



Listas Encadeadas



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Motivação
 - ▣ Estruturas de dados estáticas vs. dinâmicas
 - ▣ Limitações no uso de vetores
- Listas encadeadas simples
 - ▣ Características
 - ▣ Definição da Estrutura
 - ▣ Operações básicas
- Vantagens e desvantagens
- Exercícios

Motivação

□ Vetores

□ `int meuVetor[10];`

□ Limitações?

■ Tamanho fixo:

- quantidade de elementos não pode ser maior que a quantidade declarada

■ Disperdício de memória

- Caso todas as posições não sejam utilizadas

Motivação

- Vetores
- Alocação dinâmica resolve o problema?
 - ▣ `int* meuVetor = (int*) malloc(n*sizeof(int));`
 - ▣ Não completamente!!



- Uma vez que a memória é alocada, os mesmos problemas de limitação de espaço e desperdício de memória existirão!

Motivação

- Considere o seguinte vetor de float

1	2	3	4.5	7.6	
---	---	---	-----	-----	--

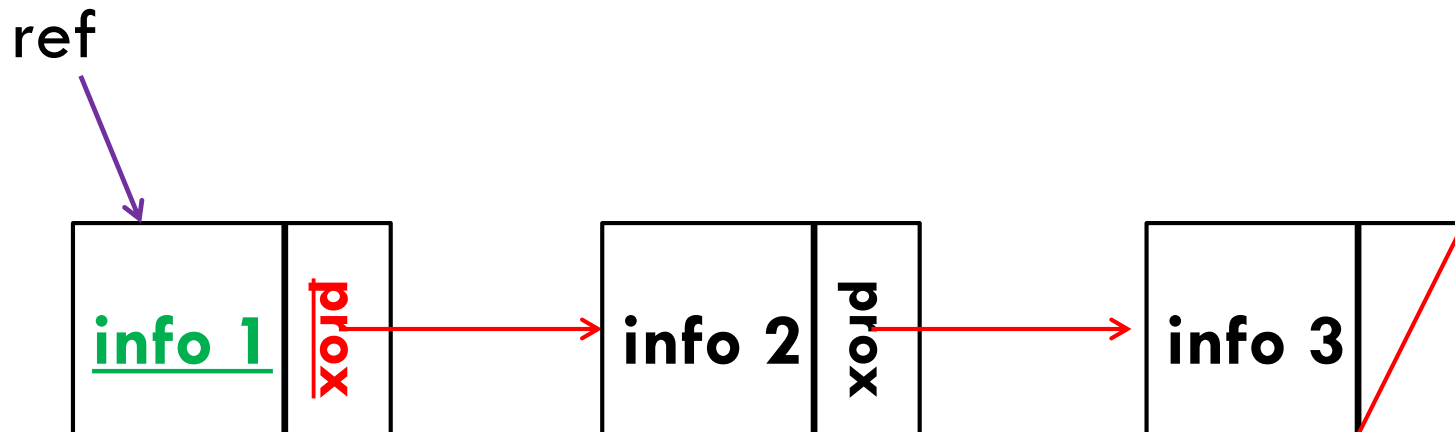
- como você faria para inserir ou remover um elemento em uma posição específica, mantendo a ordem crescente?
 - ▣ Por exemplo, inserir 2.5...
 - ▣ Ou remover o elemento 3...
 - ▣ **Problema: elementos posteriores precisam ser reorganizados**

Lista encadeada

- Resolve os problemas mencionados nos slides anteriores
 - ▣ Maior flexibilidade que vetores
 - ▣ Conjunto de dados pode crescer ou diminuir
 - ▣ Evita o desperdício de memória
 - ▣ Elementos podem ser inseridos ou removidos em posições específicas
 - Sem necessidade de reordenação do restante dos elementos

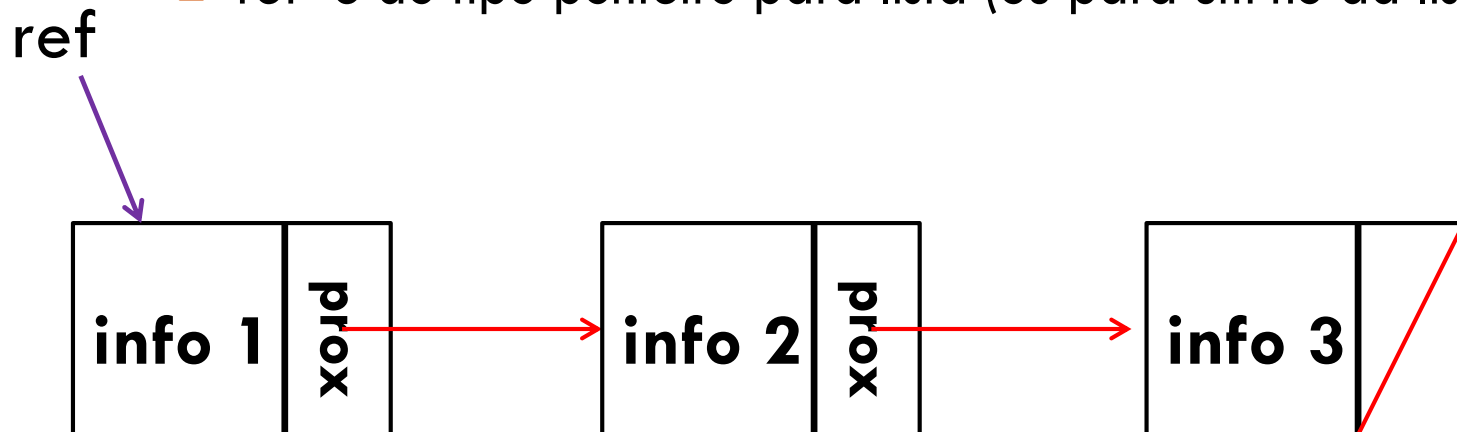
Lista encadeada

- Características principais
 - ▣ Estrutura sequencial
 - ▣ Cada elemento (ou nó) da lista possui
 - Um campo 'informação'
 - Um campo ponteiro para o próximo elemento



Lista encadeada

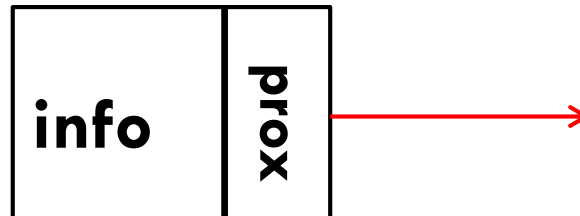
- Características principais
 - ▣ Caso não exista um próximo elemento, ponteiro 'prox' deve ser NULL
 - Indica o fim da lista
 - ▣ Existe, ainda, um ponteiro externo ('ref') que aponta para o primeiro nó da lista
 - 'ref' é do tipo ponteiro para lista (ou para um nó da lista)



Lista encadeada

- Implementação: novo tipo de dado

```
struct lista{  
    int info;  
    struct lista* prox;  
};  
typedef struct lista Lista;
```



Lista encadeada

- Implementação: criação de uma nova lista

```
Lista* criaLista()  
{  
    return NULL;  
}
```

```
int main()  
{  
    Lista* ref = criaLista();  
    ...  
}
```

- Quando a lista é criada, temos uma lista vazia:
 - Não existe nenhum elemento
 - A referência para a lista aponta para NULL
- Garante que o último elemento da lista aponta para NULL

Lista encadeada

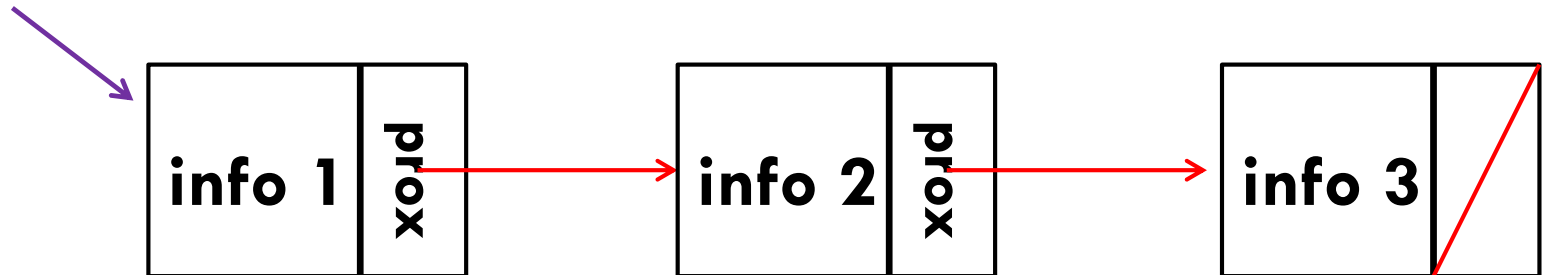
□ Implementação: Inserção

- ▣ Função deve receber a lista onde o novo elemento será inserido
- ▣ E o valor da informação do novo elemento
 - Um inteiro no nosso exemplo
- ▣ Abordagem utilizada nesta aula:
 - Inserir sempre no início da lista
 - Mais eficiente do que inserir no meio ou no final da lista

Lista encadeada

- Implementação: inserção no início da lista

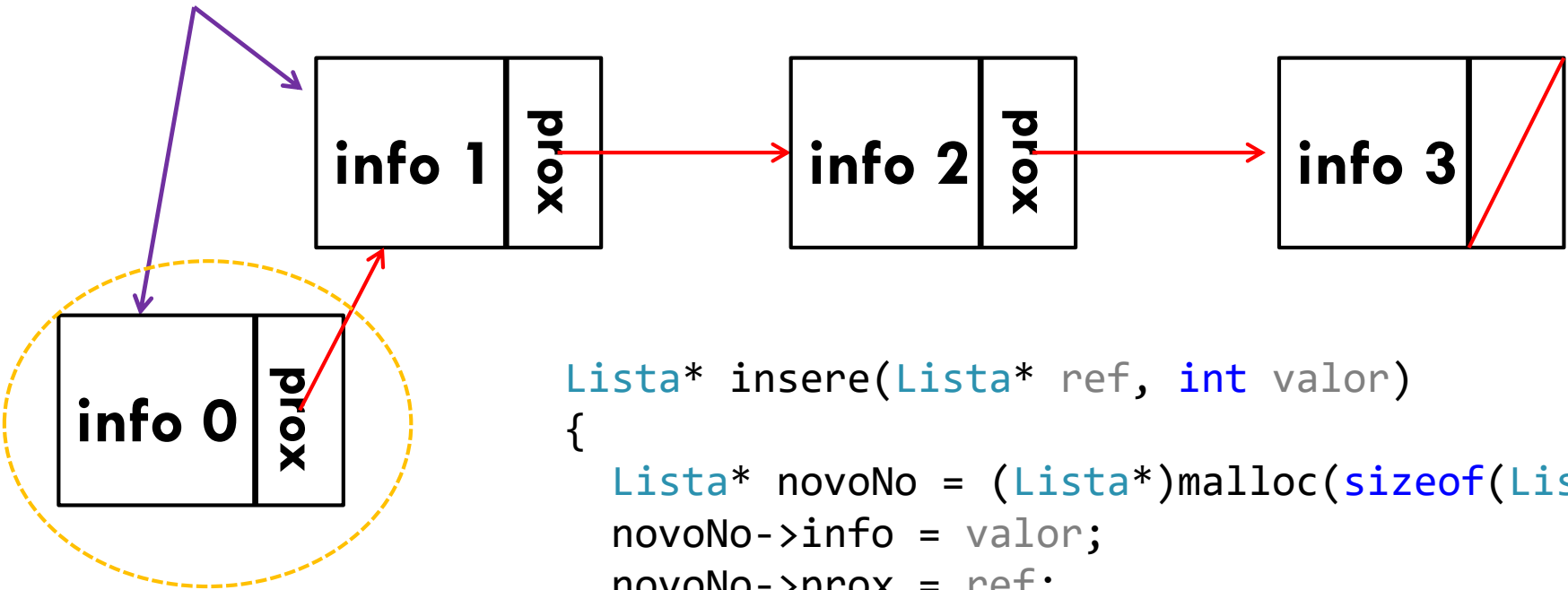
Lista* ref



Lista encadeada

- Implementação: inserção no início da lista

Lista* ref

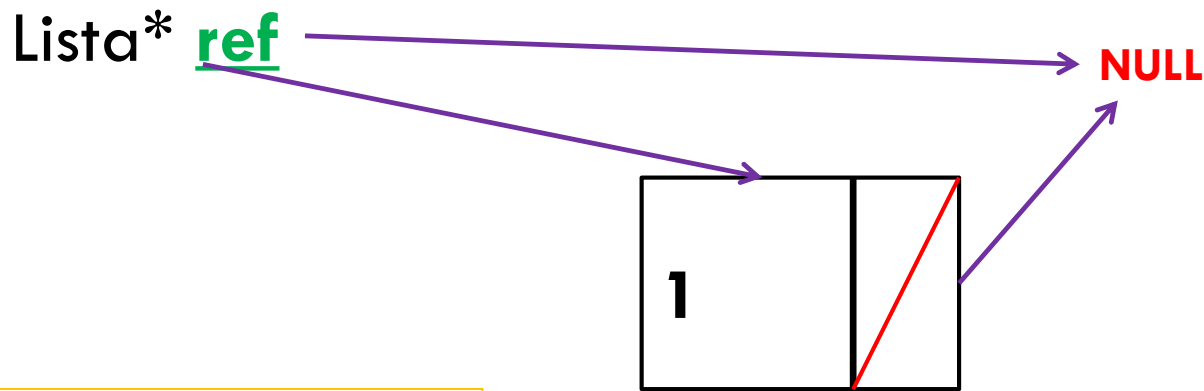


**Novo elemento
inserido!**

```
Lista* insere(Lista* ref, int valor)
{
    Lista* novoNo = (Lista*)malloc(sizeof(Lista));
    novoNo->info = valor;
    novoNo->prox = ref;
    ref = novoNo;
    return ref;
}
```

Lista encadeada

- Detalhe: cópia de 'ref' é modificada em insere()



ref é uma cópia de ref

```
Lista* insere(Lista* ref, int valor)
{
    Lista* novoNo = (Lista*)malloc(sizeof(Lista));
    novoNo->info = valor;
    novoNo->prox = ref;
    ref = novoNo;
    return ref;
}
```

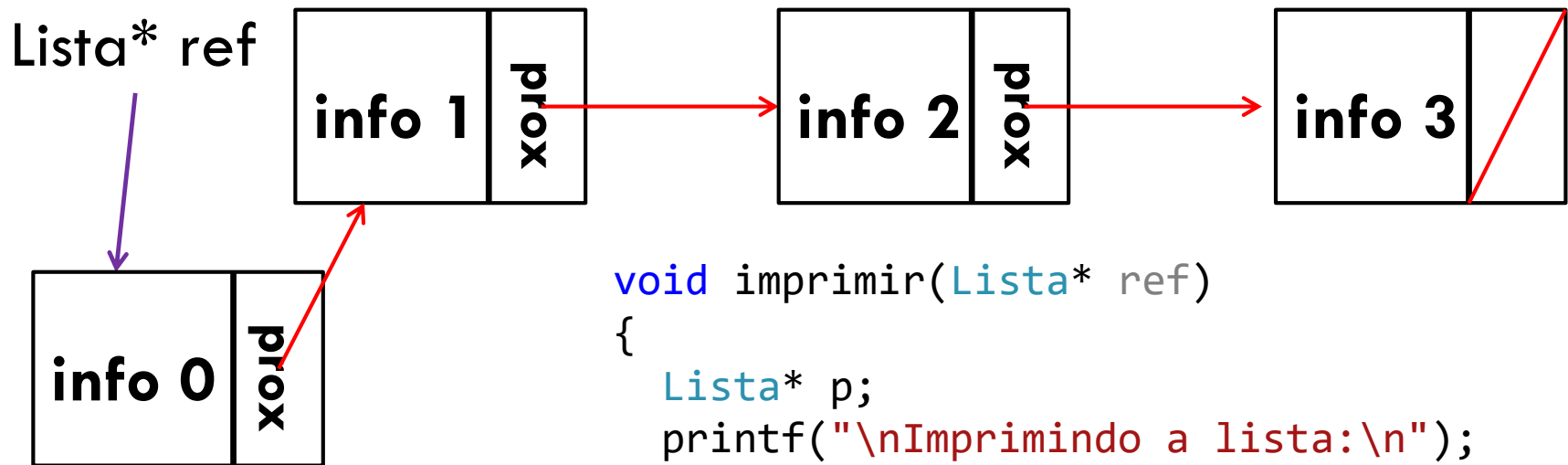
```
int main()
{
    Lista* ref = NULL;
    ref = insere(ref, 1);
    ...
}
```

Lista encadeada

- Implementação: Impressão
 - ▣ Após inserção de elementos, é importante imprimir a lista para testar se o funcionamento está correto
 - Mesmo vale após remoção de elementos
 - ▣ Cada nó da lista deve ser acessado, e sua informação impressa

Lista encadeada

□ Implementação: impressão da lista

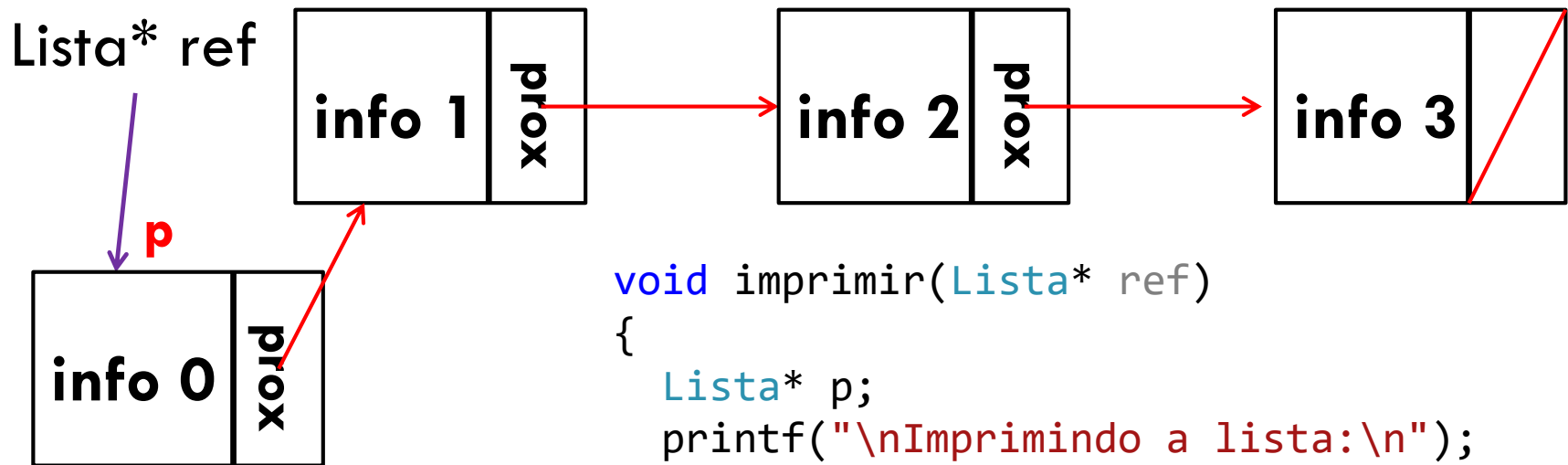


```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

p

Lista encadeada

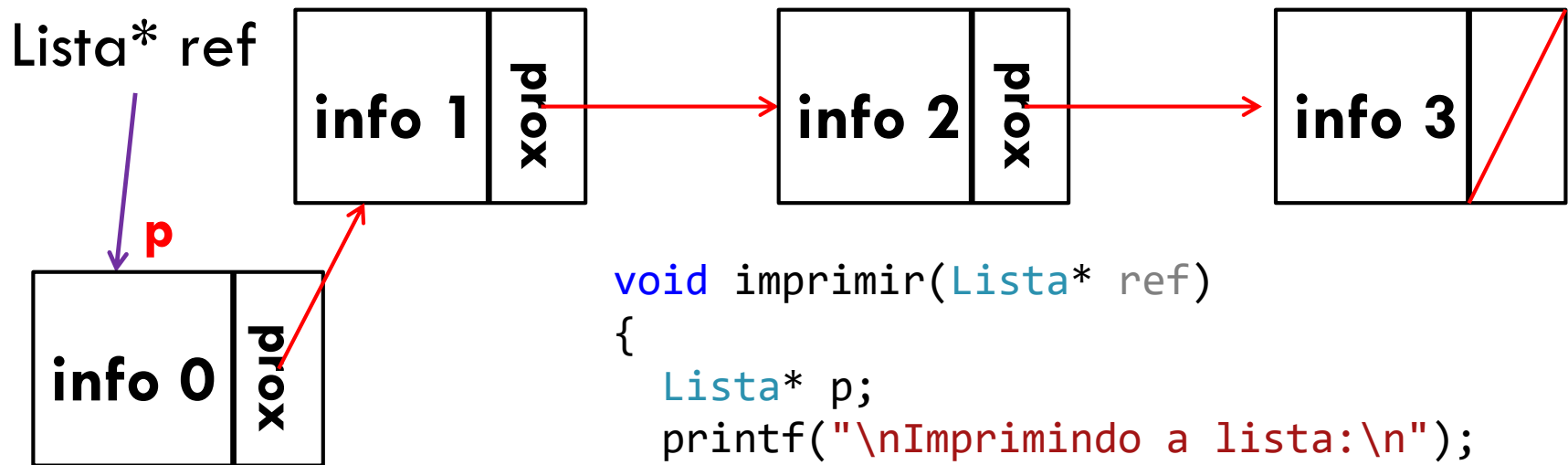
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

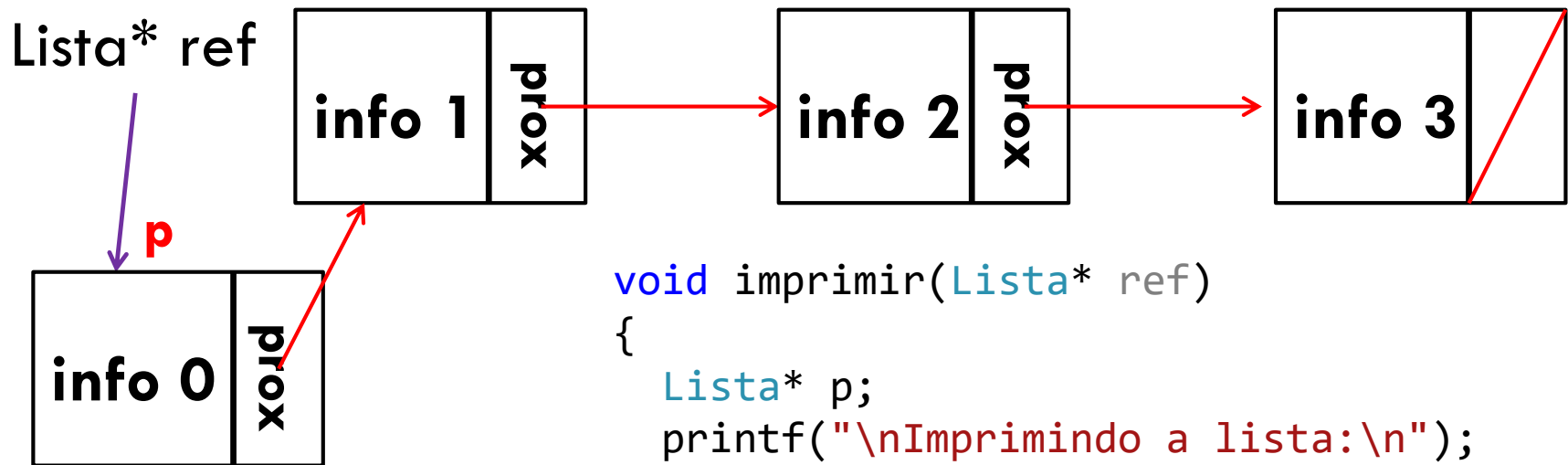
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

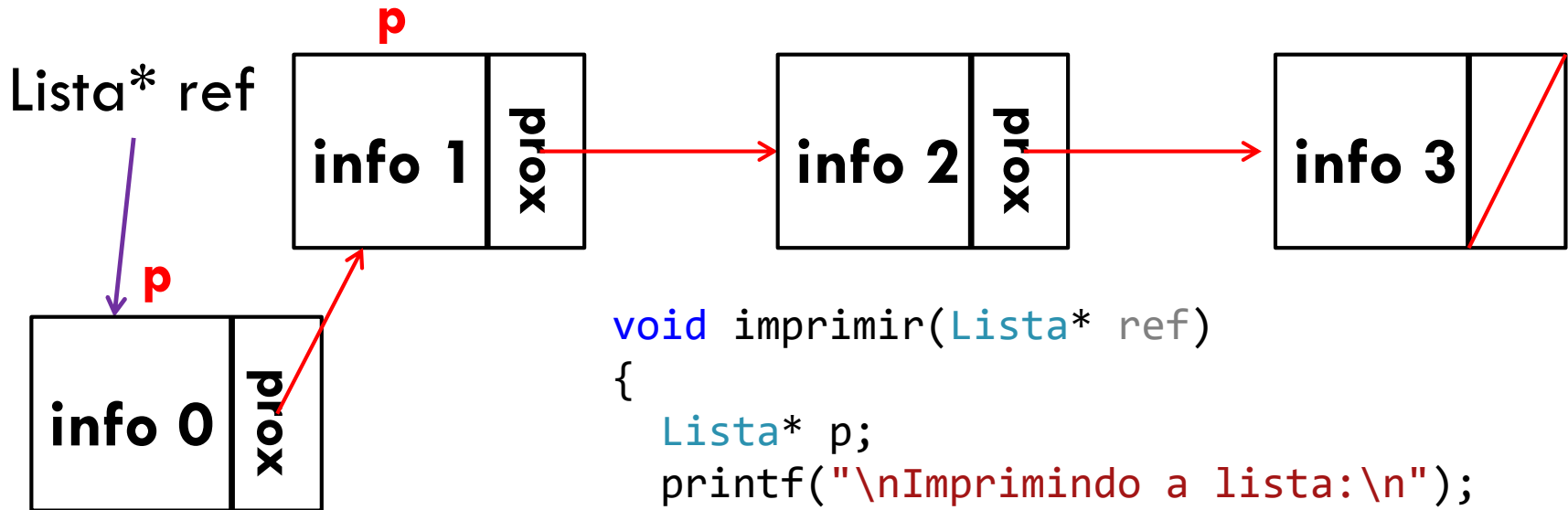
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

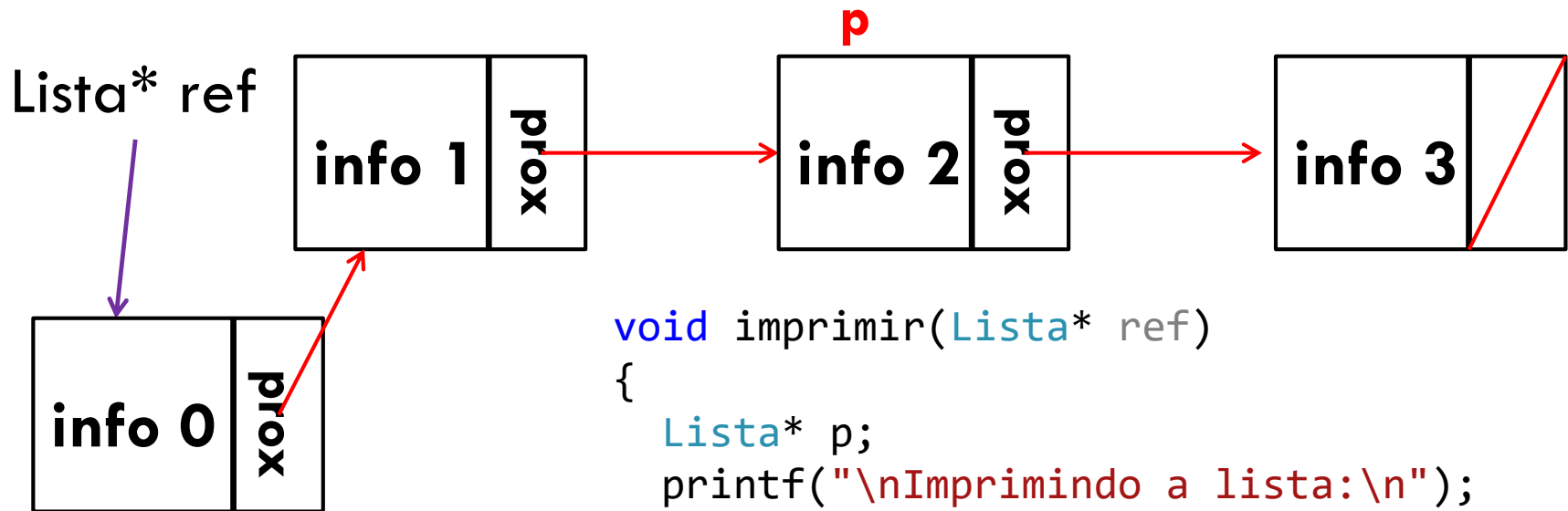
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

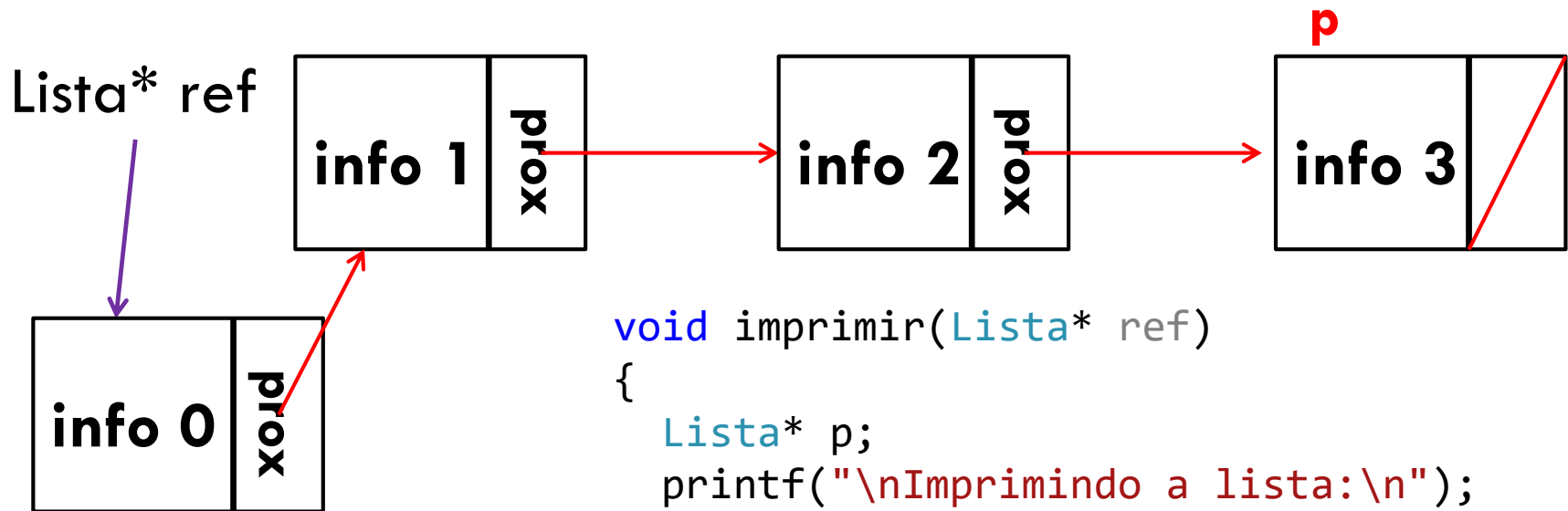
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

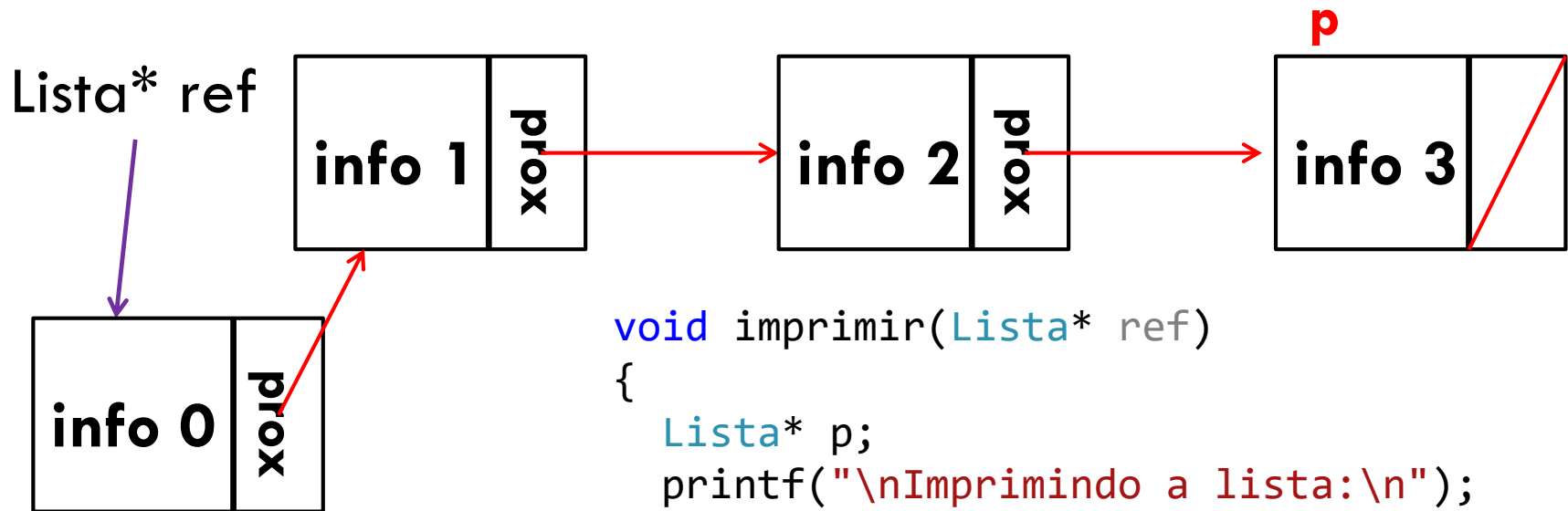
□ Implementação: impressão da lista



```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

□ Implementação: impressão da lista

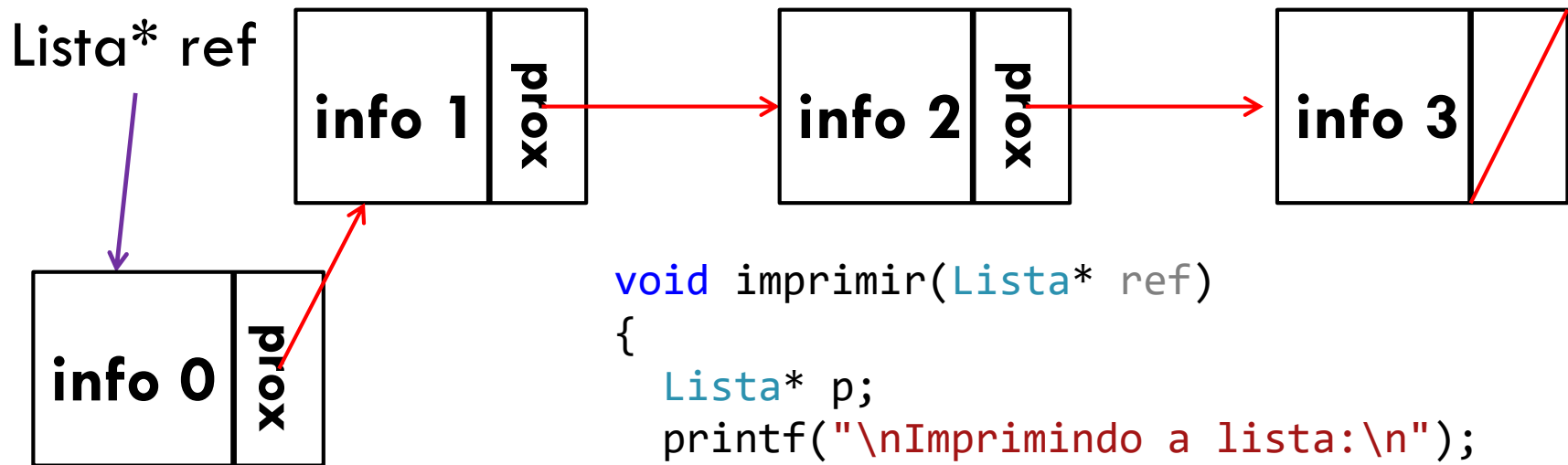


p = NULL

```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

□ Implementação: impressão da lista



p = NULL

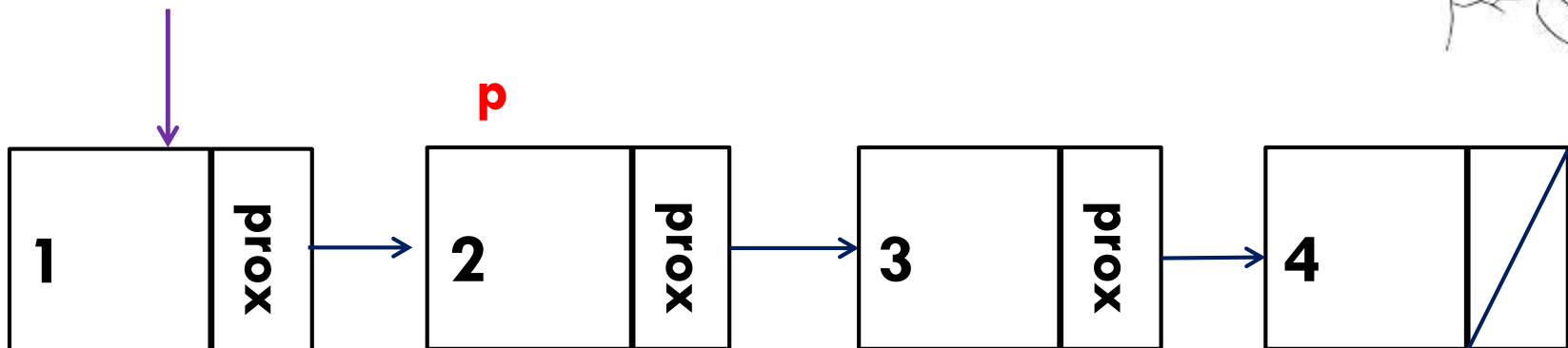
```
void imprimir(Lista* ref)
{
    Lista* p;
    printf("\nImprimindo a lista:\n");
    if (ref == NULL){
        printf("Lista vazia!");
        return;
    }
    for (p = ref; p != NULL; p = p->prox)
        printf("%d ", p->info);
}
```

Lista encadeada

❑ Remoção de elementos

- ❑ Função deve buscar um determinado elemento para ser removido
- ❑ Ponteiro 'p' percorre a lista até que o elemento a ser removido seja encontrado
 - Suponha a seguinte tentativa de remoção do nó 'p'
 - Como acessar o nó 3 se a sua referência foi perdida?

Lista* ref



Lista encadeada

□ Remoção de elementos

- ▣ Função deve buscar um determinado elemento para ser removido
- ▣ Ponteiro 'p' percorre a lista até que o elemento a ser removido seja encontrado
 - Solução para problema da referência:
 - Uso de ponteiro auxiliar 'ant', que também percorre a lista
 - 'ant' deve estar sempre em uma posição anterior a 'p'

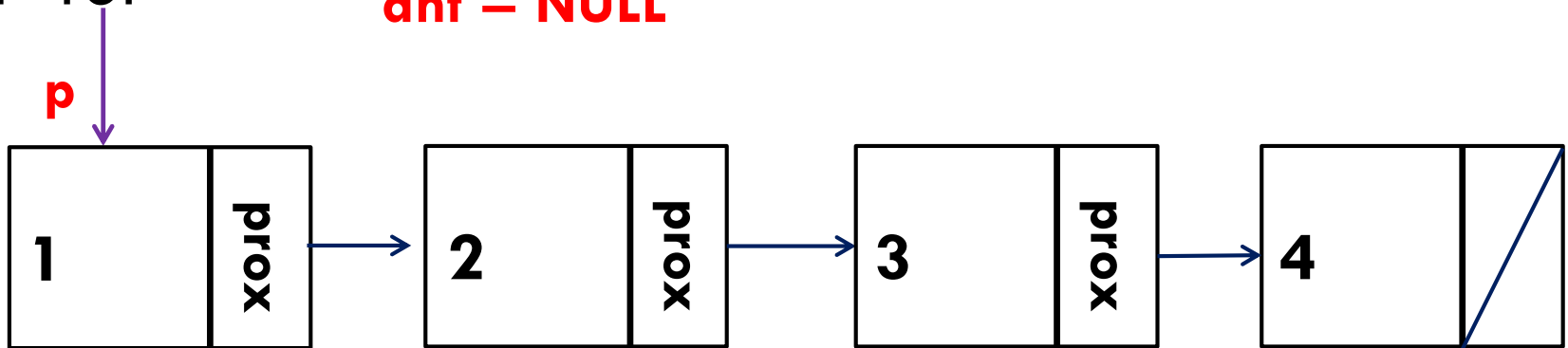
Lista encadeada

□ Remoção de elementos

▣ Ex: Remover elemento com valor 3

Lista* ref

ant = NULL



```
Lista* remove(Lista* ref, int valor)
{
    Lista* ant=NULL;
    Lista* p = ref;
```

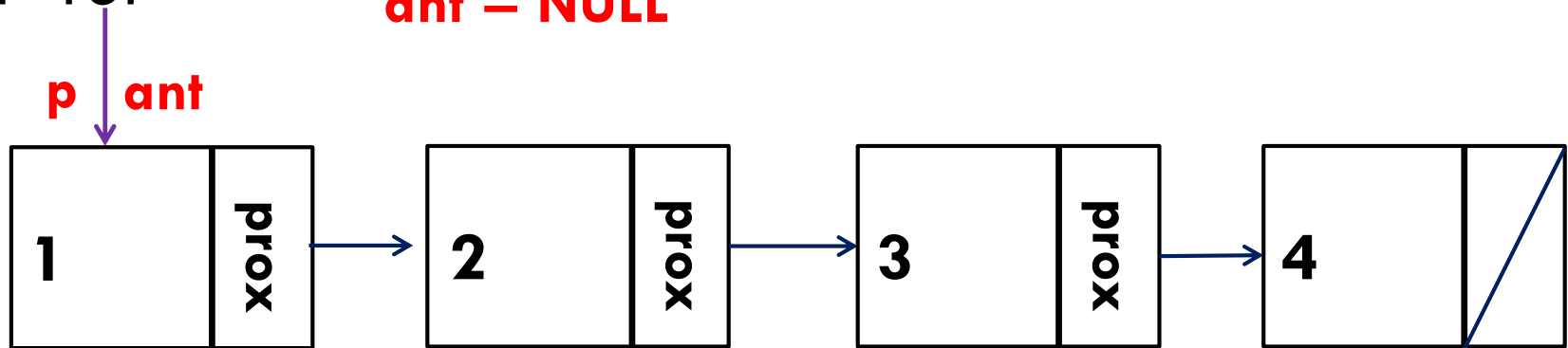
Lista encadeada

❑ Remoção de elementos

❑ Ex: Remover elemento com valor 3

Lista* ref

ant = NULL



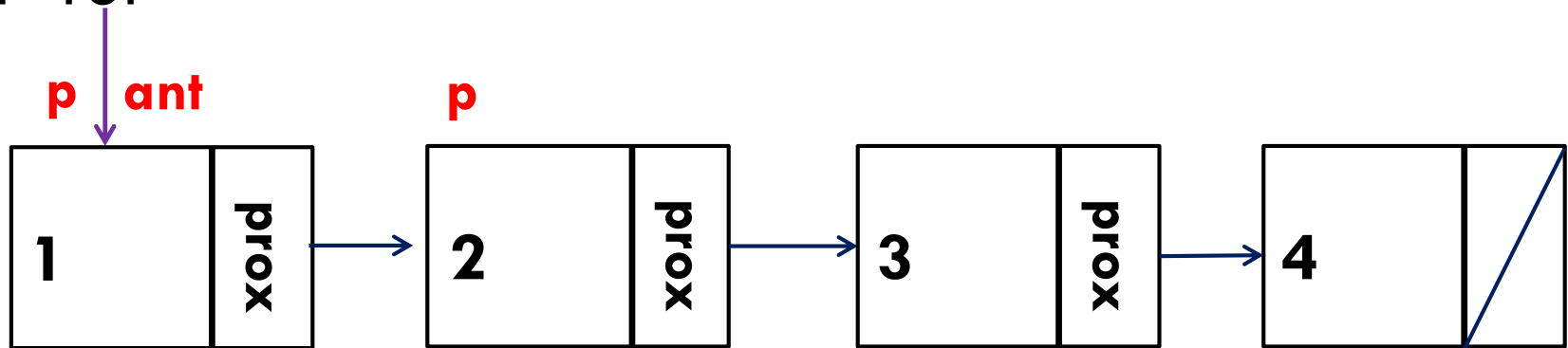
```
while (p != NULL && p->info != elemBuscado)
{
    ant = p;
    p = p->prox;
}
```

Lista encadeada

❑ Remoção de elementos

❑ Ex: Remover elemento com valor 3

Lista* ref



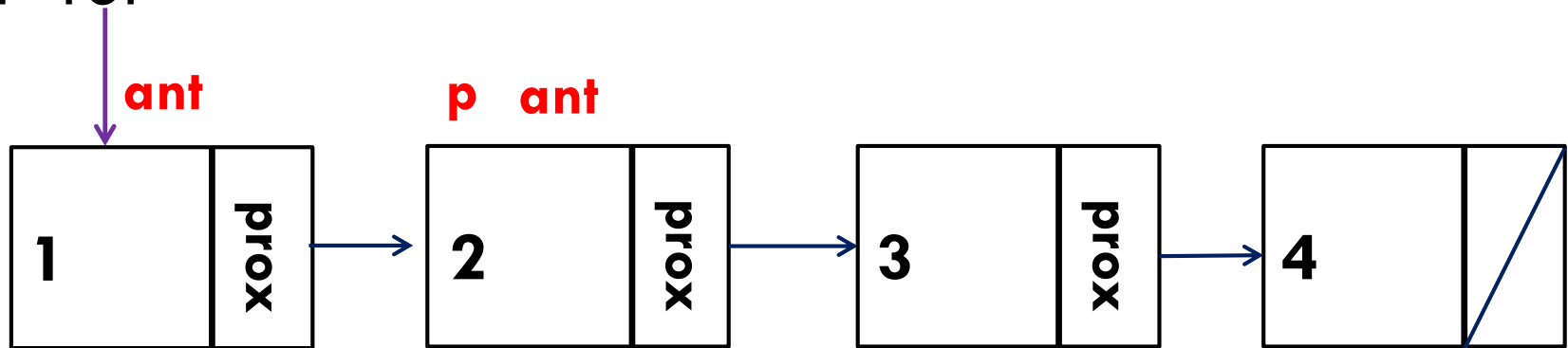
```
while (p != NULL && p->info != elemBuscado)
{
    ant = p;
    p = p->prox;
}
```

Lista encadeada

❑ Remoção de elementos

▣ Ex: Remover elemento com valor 3

Lista* ref



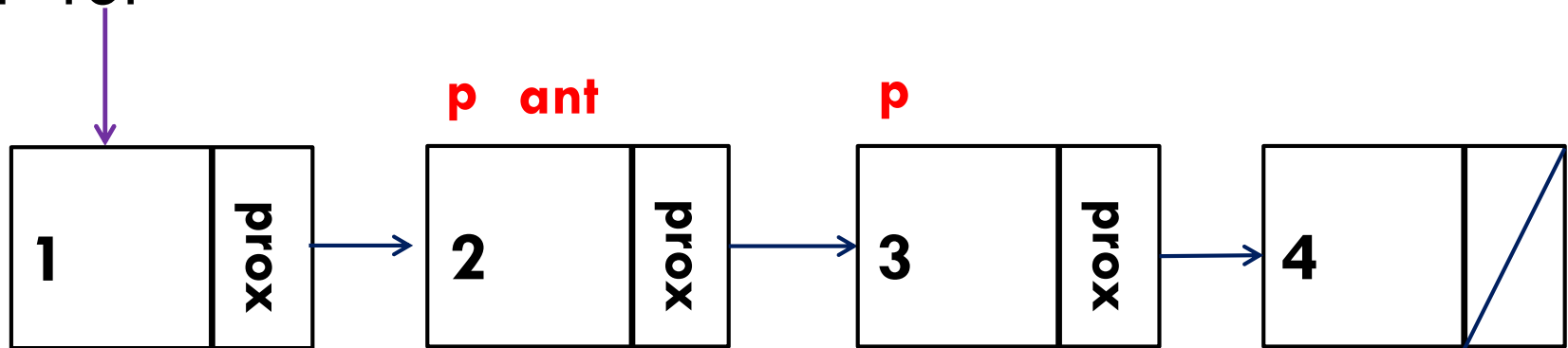
```
while (p != NULL && p->info != elemBuscado)
{
    ant = p;
    p = p->prox;
}
```

Lista encadeada

❑ Remoção de elementos

▣ Ex: Remover elemento com valor 3

Lista* ref



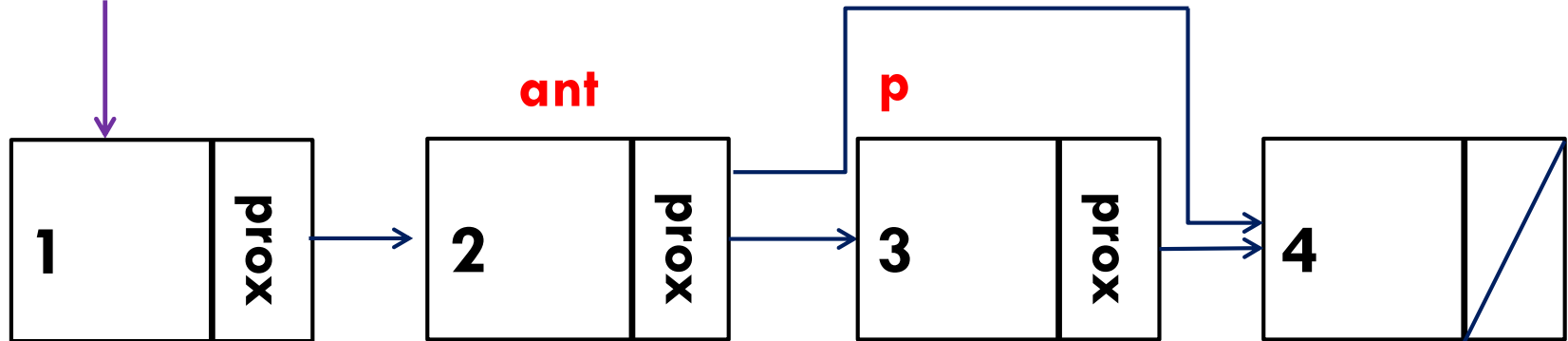
```
while (p != NULL && p->info != elemBuscado)
{
    ant = p;
    p = p->prox;
}
```

Lista encadeada

❑ Remoção de elementos

▣ Ex: Remover elemento com valor 3

Lista* ref



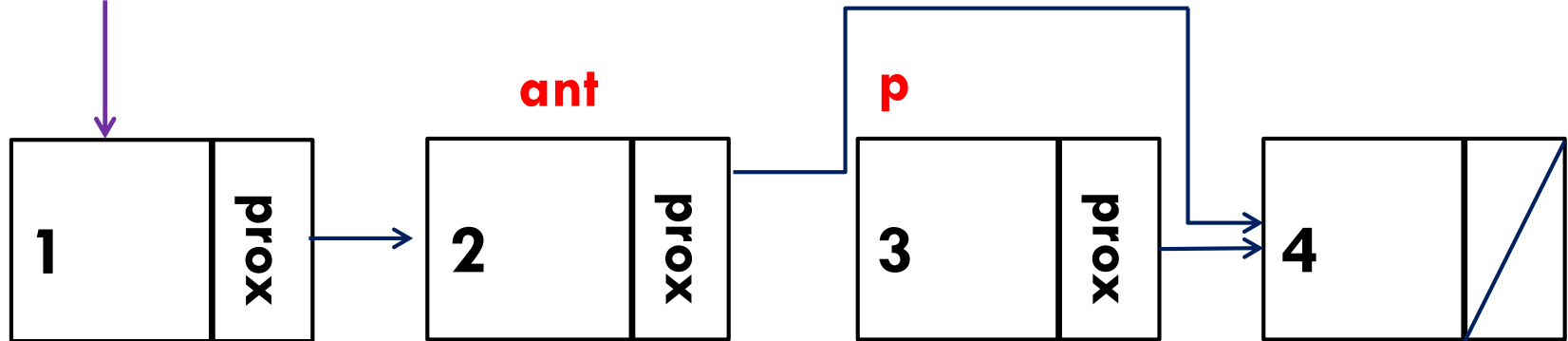
```
if (p == NULL) //elemento não encontrado
    return ref;
//remove primeiro elemento
if (p == ref)
    ref = p->prox;
else //remove do meio ou fim da lista
    ant->prox = p->prox;
```

Lista encadeada

❑ Remoção de elementos

❑ Ex: Remover elemento com valor 3

Lista* ref

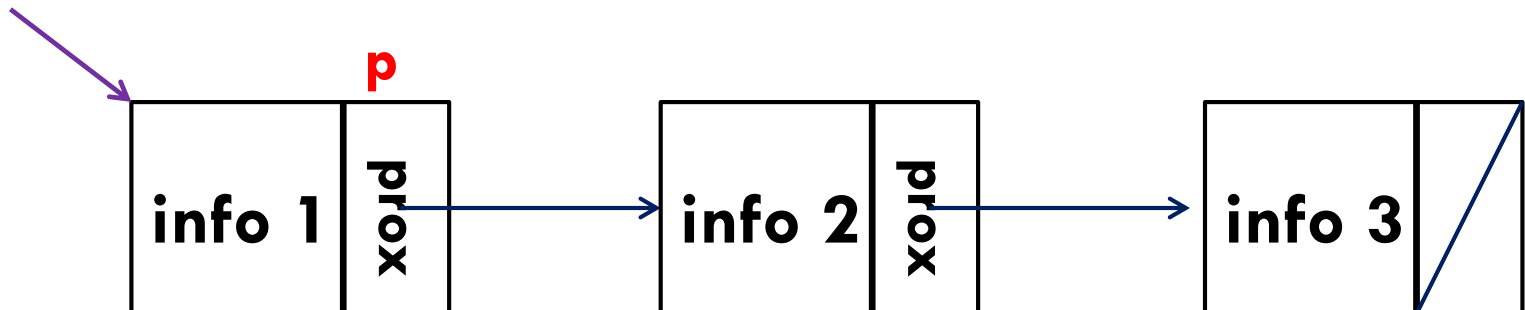


```
free(p);  
return ref;  
}
```

Lista encadeada

- Remoção de elementos
 - ▣ Caso específico: remoção do primeiro elemento da lista

Lista* ref



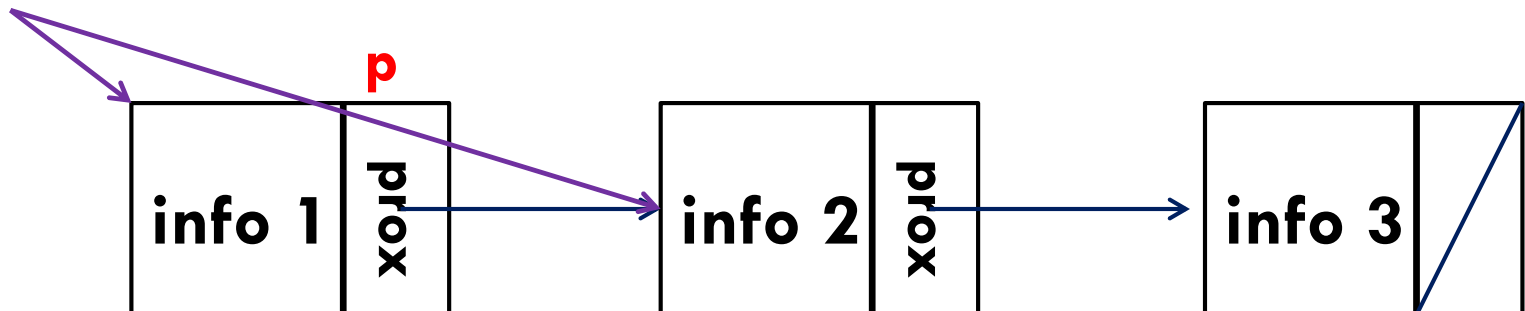
```
if (p == ref)  
    ref = p->prox;
```

Lista encadeada

□ Remoção de elementos

▣ Caso específico: remoção do primeiro elemento da lista

Lista* ref



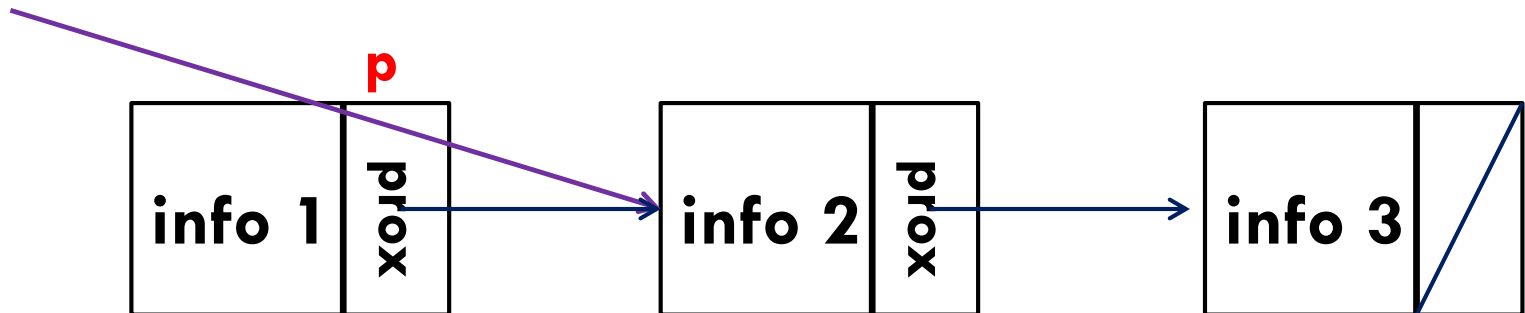
```
if (p == ref)  
    ref = p->prox;
```

Lista encadeada

- Remoção de elementos

- ▣ Caso específico: remoção do primeiro elemento da lista

Lista* ref



```
free(p);  
return ref;
```

Lista encadeada

- ❑ Liberando a memória da Lista
 - ▣ Caso específico: remoção do primeiro elemento da lista
 - ▣ Passagem de ponteiro para ponteiro
 - Alternativa para modificar ref sem ter que retornar nada

```
void libera(Lista** lista)
{
    Lista* l = *lista;

    while (l != NULL)
    {
        Lista* aux = l->prox;
        free(l);
        l = aux;
    }
    *lista = NULL;
}
```

Lista encadeada

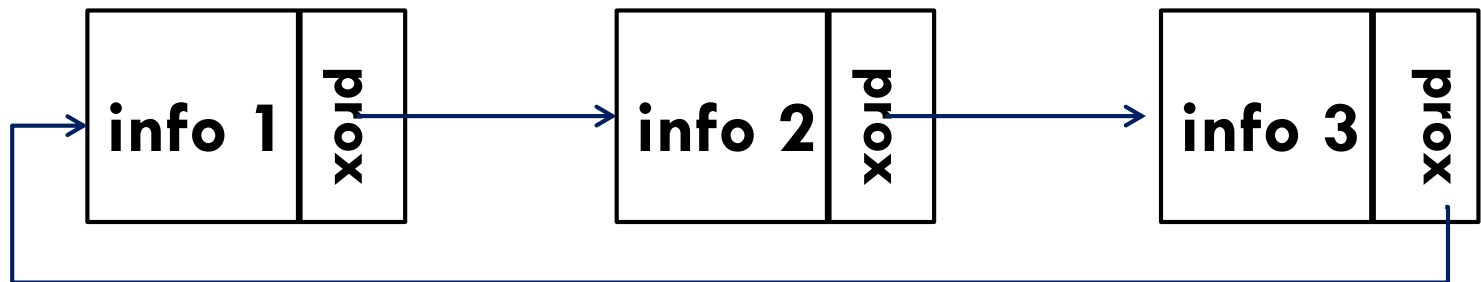
```
int main()
{
    Lista* ref = criaLista();
    ref = insere(ref, 1);
    ref = insere(ref, 2);
    ref = insere(ref, 3);
    ref = insere(ref, 4);
    imprimir(ref);  4 3 2 1
    ref = remove(ref, 2);
    imprimir(ref);  4 3 1
    ref = remove(ref, 4);
    imprimir(ref);  3 1
    ref = remove(ref, 4);
    imprimir(ref);  3 1
}
```

Lista encadeada

```
ref = remove(ref, 1);  
imprimir(ref); → 3  
ref = remove(ref, 3);  
imprimir(ref); → Lista vazia!  
ref = insere(ref, 5);  
ref = insere(ref, 6);  
imprimir(ref); → 6 5  
libera(&ref);  
imprimir(ref); → Lista vazia!  
ref = insere(ref, 7);  
ref = insere(ref, 8);  
imprimir(ref); → 8 7  
printf("\n\nFim!");  
getchar();  
return 0;  
}
```

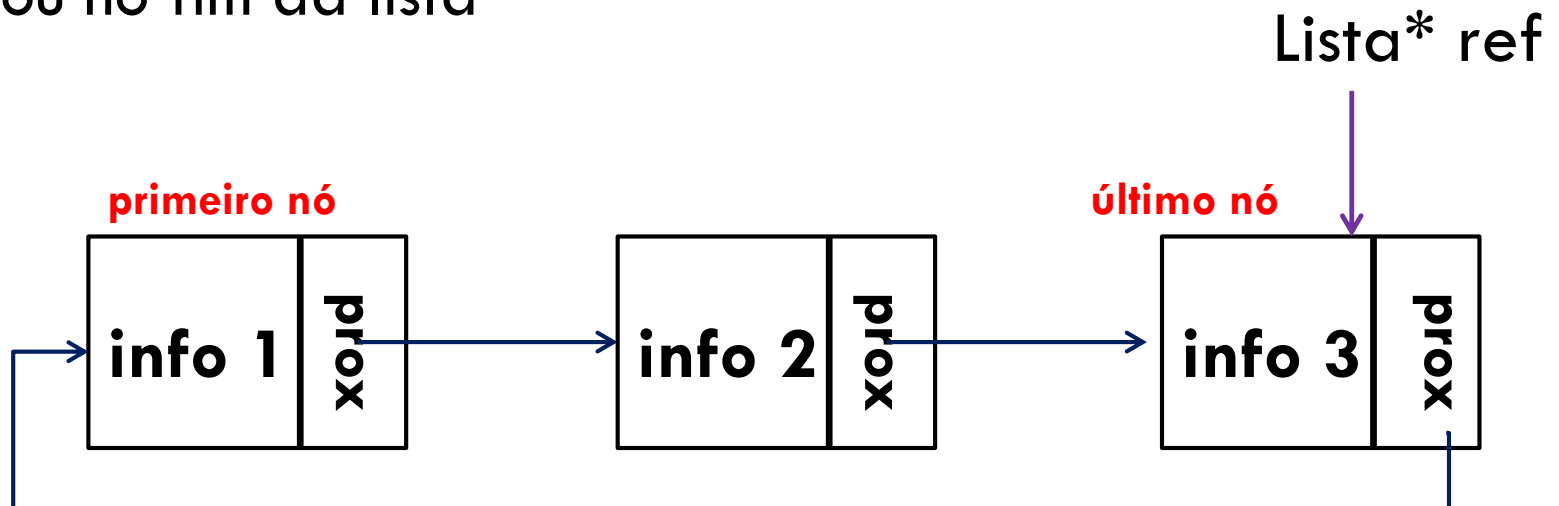
Lista encadeada circular

- Considere uma lista simples, e um ponteiro 'p' que aponta para algum dos elementos da lista
 - ▣ Problema: a partir de 'p', não podemos acessar os elementos anteriores a 'p'
 - ▣ Solução: Último nó aponta para o primeiro
 - Definição de lista circular!



Lista encadeada circular

- Não existe uma definição exata do primeiro e último elementos da lista
- No entanto, pode ser adotada uma convenção
 - ▣ Referência para lista aponta para o último elemento
 - ▣ Assim, pode-se facilmente realizar inserções no início ou no fim da lista

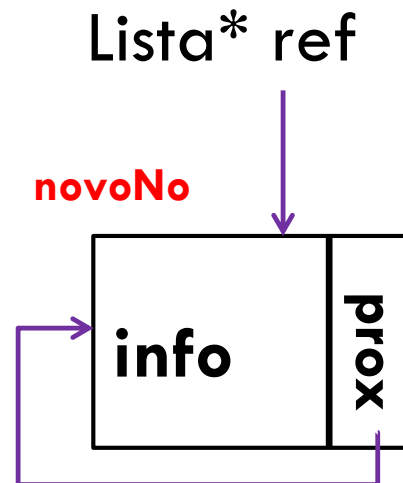


Lista encadeada circular

- Para percorrer uma lista não-circular, testamos se o elemento acessado é NULL
 - ▣ `for (aux = ref; aux != NULL; aux = aux->prox)`
 - ▣ ...
- Em uma lista circular, deve ser mantida a referência para o último nó da lista
 - ▣ A cada elemento acessado, deve ser verificado se o fim da lista foi alcançado
- Válido para busca, impressão de elementos, etc...
 - ▣ `for (aux = ref->prox; aux != ref; aux = aux->prox)`
 - ▣ ...
 - ▣ Após o laço, último nó deve ser processado!

Lista encadeada circular

- Inserção em uma lista vazia
 - ▣ Ponteiro 'próximo' deve apontar para o próprio elemento

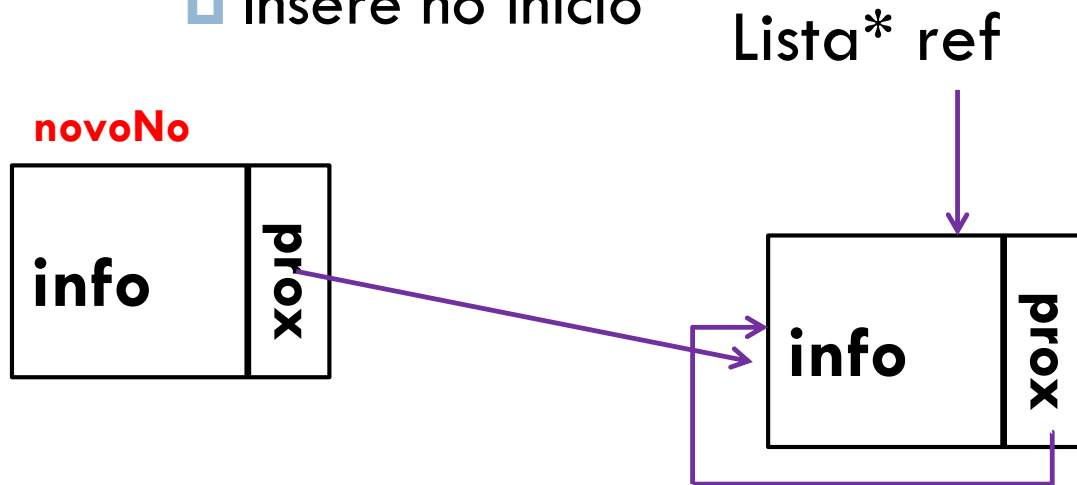


```
if (ref == NULL)
{
    novoNo->prox = novoNo;
    ref = novoNo;
}
```

Lista encadeada circular

- Inserção em uma lista contendo pelo menos um elemento

- ▣ Insere no início

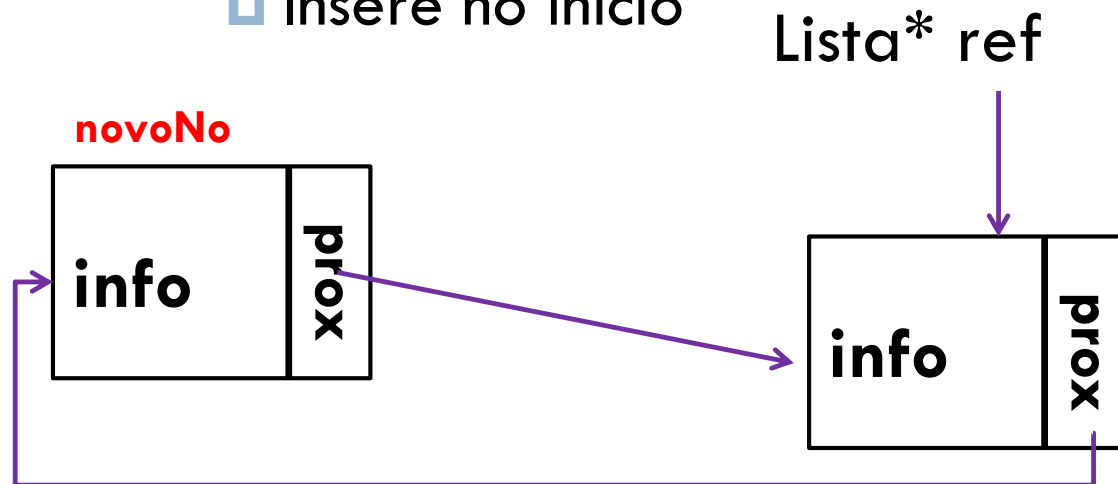


```
if (ref == NULL)
{
    novoNo->prox = novoNo;
    ref = novoNo;
}
else
{
    novoNo->prox = ref->prox;
}
```

Lista encadeada circular

- Inserção em uma lista contendo pelo menos um elemento

- ▣ Insere no início



```
if (ref == NULL)
{
    novoNo->prox = novoNo;
    ref = novoNo;
}
else
{
    novoNo->prox = ref->prox;
    ref->prox = novoNo;
}
```

Lista encadeada circular

□ Inserção em uma lista circular

```
Lista* insereCircular(Lista* ref, int info)
{
    Lista* novoNo = (Lista*)malloc(sizeof(Lista));
    novoNo->info = info;
    if (ref == NULL)
    {
        novoNo->prox = novoNo;
        ref = novoNo;
    }
    else
    {
        novoNo->prox = ref->prox;
        ref->prox = novoNo;
    }
    return ref;
}
```

Lista encadeada circular

□ Remoção de elementos

- ▣ Ponteiro 'p' percorre a lista até que o elemento a ser removido seja encontrado
 - Ponteiro auxiliar 'ant' também percorre a lista, estando sempre em uma posição anterior a 'p'
- ▣ Uma vez que o elemento é encontrado, o ponteiro de 'ant' aponta para o elemento apontado pelo ponteiro de 'p'
 - ...e 'p' é removido e sua memória liberada

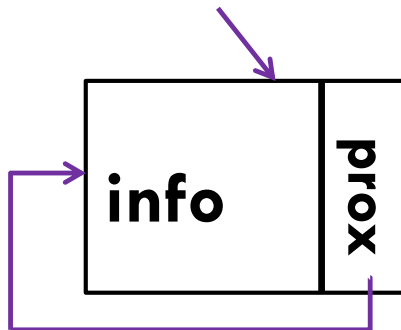
Lista encadeada circular

- Remoção de elementos
 - ▣ Ocorre de modo similar à remoção em uma lista simples
 - Porém, para encerrar busca elemento de referência é utilizado
 - No caso de listas simples, sabemos que o elemento que aponta para NULL é o último da lista

Lista encadeada circular

- Remoção de elementos
- Alguns tratamentos específicos
 - ▣ Remoção do único elemento de uma lista deve retornar NULL
 - Referencia externa para a lista recebe valor NULL, indicando que a lista se encontra vazia

Lista* ref



```
if (p->prox == p)
{
    free(p);
    return NULL;
}
```

Lista encadeada circular

- Remoção de elementos
- Alguns tratamentos específicos
 - ▣ Caso o elemento a ser removido seja a referencia da lista
 - Referência deve ser atualizada
 - Nova referência deve ser o elemento anterior

```
if (p == ref)  
    ref = ant;
```

Lista encadeada circular

□ Remoção de elementos

```
Lista* removeCircular(Lista* ref, int elemBuscado)
{
    Lista* p;
    Lista* ant;

    if (ref == NULL)
        return ref;

    p = ref->prox;
    ant = ref;
    while (p->info != elemBuscado && p != ref)
    {
        ant = p;
        p = p->prox;
    }
}
```

Lista encadeada circular

□ Remoção de elementos

```
if (p->info == elemBuscado)
{
    if (p->prox == p)
    {
        free(p);
        return NULL;
    }
    if (p == ref)
        ref = ant;
    ant->prox = p->prox;
    free(p);
    p = NULL;
}
return ref;
}
```

Lista encadeada circular

□ Impressão de elementos

```
void imprimeListaCircular(Lista* ref)
{
    Lista* aux;
    printf("\nimprimindo lista:\n");
    if (ref == NULL)
    {
        printf("\nLista vazia!\n");
        return;
    }

    for (aux = ref->prox; aux != ref; aux = aux->prox)
        printf("%d ", aux->info);
    printf("%d ", aux->info);
}
```

Mais Variações

▣ Lista duplamente encadeada



▣ Lista circular duplamente encadeada



Vantagens em relação ao uso de vetores

- ▣ Maior flexibilidade
 - Conjunto de dados pode crescer ou diminuir
- ▣ Evita o desperdício de memória
- ▣ Elementos podem ser removidos em posições específicas
 - E também inseridos em posições específicas
 - Mostre nos exercícios!

Desvantagens em relação ao uso de vetores

- ▣ Acesso sequencial

- Para alcançar o n -ésimo elemento, os ' $n-1$ ' elementos anteriores a ' n ' devem ser acessados

- ▣ Com vetores

- Acesso constante
 - `meuVetor[n]`
 - Acesso mais rápido

Exercícios

1. Crie sua própria implementação de Lista Encadeada, com as funcionalidades apresentadas nesta aula
2. Modifique a lista da questão 1 para armazenar inteiros na ordem crescente
 - ▣ Elementos podem ser inseridos no meio ou no final da lista!
 - ▣ Modifique a operação 'remove' para que seja mais eficiente do que na lista da questão 1 (somente a parte necessária da lista deve ser processada)

